

Filipino Adaptation of OmniVoice: Fine-Tuning a Multilingual Text-to-Speech Model Using the Philippine Languages Database

COSC-E4 Elective 4 - Machine Learning

May 20, 2026

Abstract

This study investigates Filipino adaptation of OmniVoice, a pretrained massively multilingual zero-shot text-to-speech model, using the Filipino subset of the Philippine Languages Database. Although OmniVoice already includes Filipino, the model reports only 7.71 hours of Filipino in its multilingual training mixture, leaving room to examine whether supervised Filipino fine-tuning improves generated speech. The methodology prepares paired Filipino audio and transcript data, filters read-speech utterances, converts the dataset into OmniVoice JSONL and tokenized WebDataset format, fine-tunes the pretrained checkpoint under Kaggle T4 GPU constraints, and evaluates generated audio against the base model. Training and evaluation runs are tracked in Weights and Biases to support hyperparameter comparison across learning rates, step counts, and checkpoint policies. Evaluation uses word error rate for intelligibility, objective speaker similarity for voice cloning quality, and UTMOS for predicted naturalness, with listening checks used to interpret metric changes. The best evaluated checkpoint is the 1e-5 learning-rate run selected by best development loss at step 4900, reducing WER from 22.55% to 18.52% while keeping objective speaker similarity slightly above the base model.

1 Introduction

Text-to-speech (TTS) and voice cloning systems generate speech waveforms from text and, in zero-shot voice cloning settings, imitate a target speaker from a short reference recording. Recent neural speech generation models increasingly use large-scale pretrained models, discrete audio tokens, and generative architectures to synthesize natural and speaker-conditioned speech (Wang et al., 2023; Le et al., 2023; Wang et al., 2024; Zhu et al., 2026). These systems are useful for accessibility, education, localization, narration, and language-learning tools. However, practical speech generation quality remains uneven across languages because high-resource languages tend to dominate available datasets and benchmarks.

Filipino is widely used in the Philippines, but open Filipino TTS and voice cloning workflows remain less mature than English-focused systems. OmniVoice is a promising starting point because it is open source, supports more than 600 languages, and includes Filipino with language ID `fil` (Zhu et al., 2026). However, language support does not automatically imply strong performance in pronunciation, intelligibility, naturalness, or speaker similarity. The OmniVoice language table lists 7.71 hours of Filipino in its training mixture, while the Philippine Languages Database (PLD) reports 48:56:36 of Filipino speech (Cajote et al., 2024). This difference motivates an adaptation study using supervised fine-tuning.

1.1 Problem Statement

Open Filipino speech generation systems face two related limitations. First, high-quality voice cloning systems are often optimized around languages with larger training corpora and stronger

evaluation coverage. Second, multilingual support in a pretrained model may still be thin for a specific low-resource language, especially when the available language-specific hours are small relative to the full training mixture.

The machine learning problem addressed in this study is:

Given a pretrained multilingual TTS model and a curated Filipino speech-text dataset, determine whether supervised fine-tuning improves Filipino speech generation compared with the original base checkpoint.

The study evaluates improvement using objective metrics for intelligibility, speaker similarity, and naturalness, then interprets those metrics with generated audio samples. The expected outcome is a reproducible fine-tuning and evaluation workflow for measuring Filipino adaptation of OmniVoice.

2 Related Works

Voice cloning is closely related to text-to-speech, but its goal is more specific: the system must generate the requested text while preserving the voice characteristics of a target speaker. Azzuni and El Saddik (2025) frame modern voice cloning around speaker adaptation, few-shot TTS, zero-shot TTS, and multilingual TTS. This distinction is important for the present study because different settings use different amounts of target data. Speaker adaptation usually updates a model using speaker or language examples, while zero-shot voice cloning tries to imitate a reference voice without updating the model at inference time. This study uses a zero-shot model as its starting point, but the research task itself is supervised adaptation: Filipino text-audio pairs are used to fine-tune the pretrained model.

Recent TTS research has moved from narrowly trained speaker-specific systems toward large pretrained speech generation models. VALL-E helped establish neural codec language modeling for TTS by generating discrete audio codec codes conditioned on text and a short acoustic prompt (Wang et al., 2023). Voicebox extended this direction with text-guided speech infilling and multilingual training, showing that one speech generation model can support several generation and editing tasks (Le et al., 2023). MaskGCT and F5-TTS further reflect the trend toward non-autoregressive generation, where systems aim to produce high-quality speech more efficiently than step-by-step autoregressive decoding (Wang et al., 2024; Chen et al., 2024). Across these works, the shared pattern is clear: modern voice cloning increasingly depends on pretrained generative models, speech tokenizers, and short speaker prompts rather than training a separate model for every speaker.

OmniVoice is the work most directly connected to this project because it combines those trends with broad language coverage. Zhu et al. (2026) present OmniVoice as a massively multilingual zero-shot TTS model with a diffusion language model-style, discrete, non-autoregressive architecture. Its main difference from two-stage systems is that it does not first predict semantic tokens and then convert them into acoustic tokens. Instead, OmniVoice directly maps text to multi-codebook acoustic tokens. The model uses full-codebook random masking during training and initializes from pretrained LLM weights to improve intelligibility. It also uses the Higgs Audio tokenizer to extract 8-codebook acoustic tokens and reconstruct speech waveforms (Zhu et al., 2026; Boson AI, 2025). These design choices make OmniVoice a practical base model for a Filipino adaptation experiment: the model already supports Filipino, but it can still be tested for improvement when exposed to more Filipino paired speech data.

The machine learning method in this project is supervised fine-tuning, which is a form of transfer learning. Transfer learning reuses knowledge learned from a source setting to improve learning in a related target setting (Pan and Yang, 2010). In this study, the source setting is OmniVoice’s pretrained multilingual speech generation model, and the target setting is Filipino

TTS and voice cloning. The supervised labels are the paired PLD transcripts and audio recordings. Fine-tuning is appropriate here because training a modern TTS model from scratch would require far more data and compute than the project constraints allow, while updating a pre-trained checkpoint can adapt the text-to-acoustic-token mapping toward Filipino pronunciation and usage under a Kaggle T4 workflow.

The dataset source is the Philippine Languages Database introduced by Cajote et al. (2024). PLD was developed as a multilingual speech corpus for Philippine spoken languages and covers Filipino, English, Cebuano, Kapampangan, Hiligaynon, Ilokano, Bikolano, Waray, and Tausug. The corpus is relevant because it provides the kind of paired speech and transcript data required for supervised TTS fine-tuning. The Filipino subset also gives this study a concrete gap to investigate: OmniVoice already lists Filipino support, but its reported Filipino hours are much smaller than the Filipino speech available in PLD. The project therefore asks whether Filipino-specific supervised adaptation improves the base model instead of assuming that multilingual support alone is sufficient.

Evaluation in recent zero-shot TTS work usually separates three concerns: intelligibility, speaker preservation, and naturalness. Following Voicebox and OmniVoice, this study uses WER or CER for intelligibility, SIM-o for objective speaker similarity, and UTMOS for predicted naturalness (Zhu et al., 2026; Le et al., 2023). WER and CER are based on edit distance, commonly associated with Levenshtein’s insertion, deletion, and substitution operations (Levenshtein, 1966). Because WER depends on the ASR model used to transcribe generated speech, this study also cites Whisper, HuBERT, and Paraformer as relevant ASR backends used in modern speech evaluation pipelines (Radford et al., 2022; Hsu et al., 2021; Gao et al., 2022). Speaker similarity depends on the embedding model, so ECAPA-TDNN and WavLM are included because OmniVoice evaluation uses WavLM-based ECAPA-TDNN speaker embeddings (Desplanques et al., 2020; Chen et al., 2021). UTMOS is used as an automatic MOS predictor for naturalness (Saeki et al., 2022). Together, these metrics support the main comparison of this project: whether fine-tuned OmniVoice produces Filipino speech that is clearer without losing speaker identity or naturalness.

3 Objectives

3.1 General Objective

The general objective is to fine-tune and evaluate a pretrained multilingual TTS model for Filipino speech generation using the Philippine Languages Database.

3.2 Specific Objectives

1. Prepare a clean Filipino read-speech dataset from the Philippine Languages Database.
2. Convert the prepared dataset into the JSONL and tokenized WebDataset format required by OmniVoice.
3. Fine-tune the pretrained `k2-fsa/OmniVoice` checkpoint on Filipino speech using Kaggle T4 GPUs.
4. Track training and evaluation runs in Weights and Biases for hyperparameter comparison across learning rates, training steps, and checkpoint strategies.
5. Generate Filipino speech samples from both the base and fine-tuned models using matched test prompts and reference voices.
6. Evaluate both models using intelligibility, speaker similarity, and naturalness metrics.

7. Interpret whether Filipino-specific fine-tuning improves generated Filipino speech under limited compute constraints.

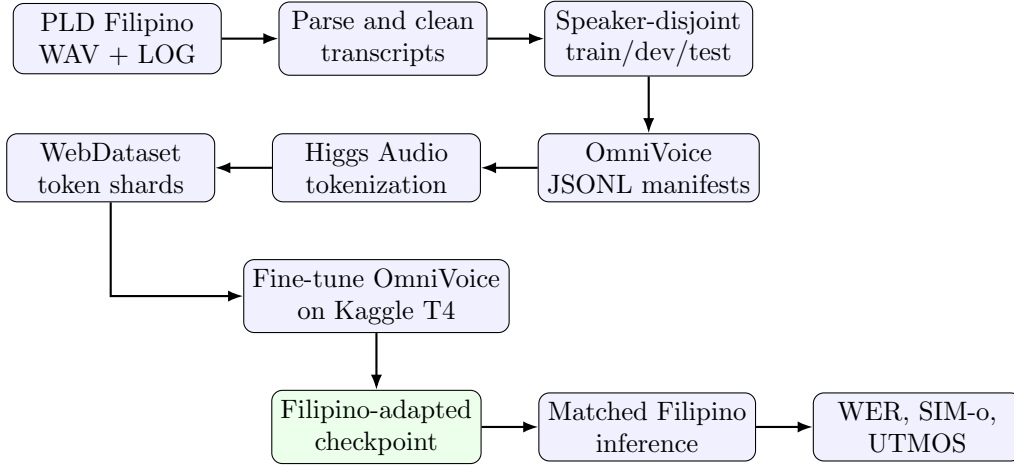
4 System Design and Methodology

4.1 System Architecture

The system has five major stages: dataset preparation, audio tokenization, model fine-tuning, inference, and evaluation.

Figure 1

Fine-tuning and evaluation system architecture.



4.2 Dataset Preparation

The primary dataset is the UP-DSP Philippine Languages Database released through Mozilla Data Collective. The corpus contains WAV audio recordings and LOG transcript files. The Filipino subset statistics from the PLD paper are shown in Table 1. The Filipino subset is large enough for a supervised adaptation experiment and contains substantially more Filipino speech than the 7.71 Filipino hours listed in OmniVoice’s original multilingual training mixture.

Table 1

Filipino subset statistics from the PLD paper (Cajote et al., 2024).

Field	Value
Language	Filipino
Language ID	<code>fil</code>
Speakers	135
Utterances	52,879
Duration	48:56:36
Average utterance length	3.5796 seconds

Table 2*PLD language statistics table excerpt.*

Language	Gender	Speaker Count	Utterance Count	Audio Duration		Tokens	
				Total (h:m:s)	Average (s)	Total	Unique
Bicolano (bik)	F	121	39,260	60:55:58	5.5873	321,721	16,049
	M	85	27,684	40:17:36	5.2397	206,642	14,631
	all	206	66,944	101:13:35	5.4436	528,363	17,005
Cebuano (ceb)	F	86	34,956	35:47:01	3.6852	144,882	8,026
	M	66	27,477	27:44:58	3.6357	114,563	7,267
	all	152	62,433	63:31:59	3.6634	259,445	6,844
English (eng)	F	23	3,156	4:31:30	5.1617	29,376	4,363
	M	7	888	1:01:40	4.1675	8,050	1,483
	all	30	4,044	5:33:11	4.9434	37,426	4,729
Filipino (fil)	F	79	30,617	31:43:55	3.7311	205,346	10,861
	M	56	22,262	20:50:48	3.3712	138,088	7,994
	all	135	52,879	48:56:36	3.5796	343,434	11,481
Hiligaynon (hil)	F	48	17,079	21:49:43	4.6012	99,087	5,397
	M	43	14,908	19:21:58	4.6766	84,676	4,906
	all	91	31,987	41:11:42	4.6363	183,763	5,767
Ilokano (ilo)	F	64	15,429	25:46:37	6.0145	131,316	11,603
	M	60	14,513	25:44:43	6.3862	130,500	11,642
	all	124	29,942	51:31:20	6.1947	261,816	13,270
Kapampangan (pam)	F	104	35,024	49:42:02	5.1086	225,595	12,827
	M	83	26,926	40:37:22	5.4313	176,947	12,629
	all	187	61,950	90:19:25	5.2488	402,542	14,221
Pangasinan (pag)	F	12	3,959	6:01:36	5.4802	24,819	4,302
	M	6	1,945	3:07:00	5.7687	11,698	3,148
	all	18	5,904	9:08:36	5.5753	36,517	4,773
Tausug (tsg)	F	4	1,185	1:43:09	5.2236	12,684	2,023
	M	9	2,103	3:06:34	5.3233	7,279	1,536
	all	13	3,288	4:49:45	5.2874	19,963	2,376
Waray-Waray (war)	F	48	15,337	22:51:12	5.3643	94,764	6,071
	M	26	8,500	12:04:10	5.1118	52,704	5,518
	all	74	23,837	34:55:23	5.2743	147,468	6,291
Total	-	1,030	343,208	454:49:43	4.7708	2,220,737	-

Table 2: Summary statistics for the Philippine Languages Database. Below the each Philippine language name is its language ID in parenthesis, based from the ISO 639-3 standard, as published in Ethnologue (Eberhard et al., 2024) Note that the total token and unique token counts do not include yet the transcripts from the spontaneous speech part as this part of the corpora is still being transcribed.

Only Filipino read speech is used in the initial experiment. Read speech is preferred because each utterance is expected to match a prepared transcript, which is important for supervised TTS fine-tuning. Spontaneous speech is excluded because a speaker’s response may not match the displayed prompt, increasing transcript-audio mismatch risk.

The data preparation script parses PLD files and writes OmniVoice-compatible JSONL rows. Each row contains an utterance ID, relative audio path, transcript, and language ID.

```
{
  "id": "0000.110816.021250.0001",
  "audio_path": "wavs/0000/0000.110816.021250.0001.wav",
  "text": "manugang",
  "language_id": "fil"
}
```

The cleaned local package is structured as follows:

```
data/pld_filipino_clean/
```

```
wavs/  
train.jsonl  
dev.jsonl  
test.jsonl
```

The current split is shown in Table 3. Speakers are disjoint across train, development, and test splits to reduce speaker leakage between training and evaluation.

Table 3

Prepared Filipino dataset split.

Split	Samples	Percent	Hours	Speakers
Train	33,448	79.12%	31.902	103
Development	4,506	10.66%	3.705	13
Test	4,322	10.22%	4.165	13
Total	42,276	100.00%	39.772	129

The cleaning process keeps read Filipino speech, removes spontaneous prompt sources, excludes transcripts containing parentheses or digits, removes unreadable or zero-byte WAV files, keeps clips between 1.0 and 15.0 seconds, and applies light transcript normalization. The filtering is intentionally conservative because TTS fine-tuning is sensitive to noisy text-audio alignment.

4.3 Model Selection and Adaptation

The selected model is `k2-fsa/OmniVoice`. OmniVoice was chosen because it is open source, supports Filipino, provides pretrained checkpoints, supports voice cloning, and includes official fine-tuning and evaluation scripts. The study uses supervised fine-tuning and transfer learning. The base OmniVoice model has already learned multilingual speech generation from a large corpus, while the Filipino PLD subset provides paired Filipino text and audio examples for adapting the model’s text-to-acoustic-token mapping.

Figure 2
OmniVoice paper first page.

arXiv:2604.00688v3 [cs.CL] 21 Apr 2026



The main model components in this study are:

- **Input data:** Filipino text transcripts, Filipino audio, optional reference voice audio, and language ID `fil`.
- **Training target:** 8-codebook acoustic token sequences extracted from Filipino WAV files.
- **Learning method:** supervised fine-tuning of a pretrained neural TTS model.
- **Baseline:** original `k2-fsa/OmniVoice` checkpoint without Filipino-specific fine-tuning.
- **Evaluation:** matched comparison of generated audio from base and fine-tuned checkpoints.

4.4 Audio Tokenization

OmniVoice does not train directly on raw waveforms. Its data preparation flow converts each audio file into discrete audio tokens and stores those tokens in WebDataset shards. The official OmniVoice documentation states that audio files are resampled to 24 kHz and converted into 8-layer discrete tokens using the configured audio tokenizer. The default tokenizer path in the OmniVoice examples is `eustlb/higgs-audio-v2-tokenizer`, and the OmniVoice paper describes the Higgs Audio tokenizer as the component used to extract 8-codebook acoustic tokens and reconstruct audio (Zhu et al., 2026; Boson AI, 2025).

This tokenization step is central to the learning problem. A waveform is converted from a continuous signal into a matrix of acoustic token IDs. Each row corresponds to one codebook layer, and each time step represents a short audio frame. In OmniVoice, the model is trained to recover masked acoustic tokens conditioned on text tokens, language information, prompt audio tokens, and any unmasked target tokens. During inference, the model predicts target acoustic tokens through iterative unmasking, and the Higgs Audio decoder reconstructs waveform audio from the generated token matrix.

For this study, tokenization is performed once and reused as a Kaggle dataset artifact when possible. This avoids repeatedly spending GPU and disk time on audio token extraction during hyperparameter tuning.

4.5 Fine-Tuning and Experiment Tracking

Fine-tuning is performed in Kaggle using T4 GPU notebooks. Because the available GPU memory is limited, the setup uses FP16 precision, SDPA attention, smaller token batches, gradient accumulation, and checkpointing. ZeRO-style memory optimization is used when necessary to reduce optimizer-state memory pressure during training (Rajbhandari et al., 2019). Training starts from `k2-fsa/OmniVoice` rather than random initialization.

Weights and Biases is used for experiment tracking (Biewald, 2020). Each training run logs configuration values, training loss, development loss, checkpoints, generated samples, and evaluation artifacts. Hyperparameter tuning compares learning rates, step counts, and checkpoint policies, including shorter 1000-step runs, 2000-step runs, and longer runs that select the best checkpoint by development loss. The logged runs make it possible to compare whether lower development loss corresponds to better WER, SIM-o, UTMOS, and listening quality.

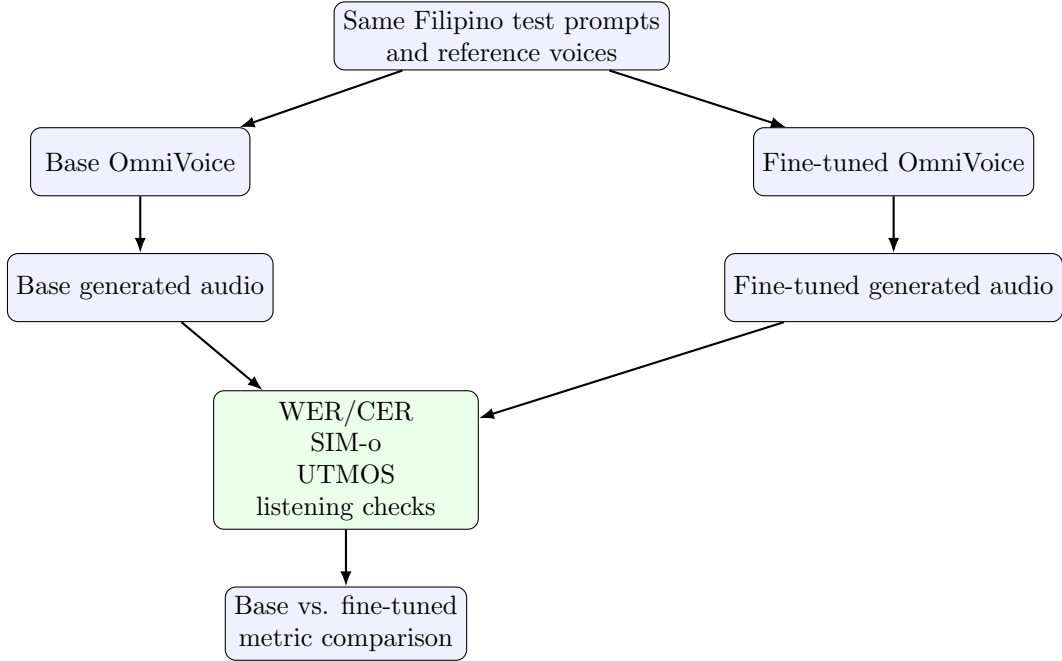
Table 4

Fine-tuning setup.

Component	Configuration
Base model	<code>k2-fsa/OmniVoice</code>
Dataset	Clean Filipino PLD read speech
Training platform	Kaggle notebook
GPU	Kaggle T4 GPU environment
Precision	FP16
Attention	SDPA
Audio tokenizer	Higgs Audio V2 tokenizer
Logging	Weights and Biases
Tuning variables	Learning rate, training steps, checkpoint policy
Checkpoint strategy	Step checkpoints and best development-loss checkpoint

4.6 Evaluation Design

The evaluation compares the base OmniVoice model and Filipino fine-tuned checkpoints on the same test prompts and reference voices.

Figure 3*Evaluation design for comparing model outputs.*

4.7 Evaluation Metrics

The evaluation uses three objective metrics from modern zero-shot TTS evaluation: WER/CER for intelligibility, SIM-o for speaker similarity, and UTMOS for naturalness. Because each metric depends on a particular evaluator model or preprocessing pipeline, all evaluation runs use the same test split, inference settings, evaluator checkpoints, and text normalization before comparing base and fine-tuned results.

4.7.1 Word Error Rate and Character Error Rate

Word error rate (WER) measures whether the generated speech communicates the intended text. The generated waveform is first transcribed by an automatic speech recognition (ASR) model. The ASR transcript is then compared with the ground-truth target text using edit-distance operations: substitutions (S), deletions (D), and insertions (I). These edit operations trace back to Levenshtein distance (Levenshtein, 1966). WER is computed as:

$$\text{WER} = \frac{S + D + I}{N} \times 100, \quad (1)$$

where N is the number of words in the reference text. Lower WER indicates that the ASR system recognized the generated speech as closer to the intended sentence. Character error rate (CER) follows the same idea but computes edit operations over characters instead of words, which can be useful for languages or text-normalization settings where word segmentation affects the metric.

In TTS evaluation, WER is not just a mathematical string distance; it is also dependent on the ASR model used to transcribe generated audio. The OmniVoice evaluation documentation reports different WER backends by benchmark: HuBERT-based WER for LibriSpeech-PC, Whisper for English Seed-TTS, Paraformer for Chinese Seed-TTS, Omnilingual-ASR for FLEURS, and Whisper or Paraformer for MiniMax multilingual evaluation depending on language (Zhu et al., 2026; Hsu et al., 2021; Radford et al., 2022; Gao et al., 2022). For this Filipino evaluation, the notebook uses the OmniVoice MiniMax multilingual WER path with

Whisper-large-v3 for the non-Chinese Filipino setting and logs insertion, deletion, substitution, and total-word counts. This makes WER useful for comparing base and fine-tuned Filipino outputs, but the result should still be interpreted as ASR-mediated intelligibility rather than a direct human comprehension score.

4.7.2 Objective Speaker Similarity

SIM-o measures whether generated speech preserves the voice identity of the reference prompt. Voicebox popularized reporting objective similarity alongside listener-based speaker similarity for zero-shot TTS evaluation (Le et al., 2023). OmniVoice follows the same evaluation family and reports SIM-o as an objective speaker-similarity metric (Zhu et al., 2026).

Computationally, SIM-o is the cosine similarity between speaker embeddings extracted from the reference audio and generated audio:

$$\text{SIM-o}(r, g) = \frac{\mathbf{e}_r \cdot \mathbf{e}_g}{\|\mathbf{e}_r\| \|\mathbf{e}_g\|}, \quad (2)$$

where $\mathbf{e}_r$ is the speaker embedding of the reference prompt and $\mathbf{e}_g$ is the speaker embedding of the generated speech. Higher values indicate that the generated speech is closer to the reference speaker in the embedding space.

The reliability of SIM-o depends on the speaker encoder. The OmniVoice evaluation path uses WavLM-based ECAPA-TDNN speaker embeddings. ECAPA-TDNN is a speaker verification architecture that improves TDNN-style speaker embeddings with channel attention, propagation, and aggregation mechanisms (Desplanques et al., 2020). WavLM provides self-supervised speech representations that are useful beyond ASR, including speaker-related tasks (Chen et al., 2021). In this study, SIM-o is used to check whether Filipino fine-tuning improves intelligibility without damaging voice cloning identity. A fine-tuned model that reduces WER but lowers SIM-o substantially may be clearer but worse at preserving the reference speaker.

4.7.3 UTMOS

UTMOS estimates speech naturalness using a learned mean opinion score (MOS) prediction model. Saeki et al. (2022) introduced UTMOS as the UTokyo-SaruLab system for the Voice-MOS Challenge 2022, where the goal was to predict human MOS ratings for speech samples. Unlike WER, which evaluates textual intelligibility, and SIM-o, which evaluates speaker identity, UTMOS estimates whether the generated waveform sounds natural or human-like.

In the evaluation pipeline, each generated audio file is passed to the UTMOS model, and the resulting predicted MOS values are averaged for each system. Higher UTMOS indicates better predicted naturalness. However, UTMOS is still a predictor trained from listening-test data, so it should not be treated as a complete replacement for human listening checks. For this reason, UTMOS is interpreted together with WER, SIM-o, and selected Filipino listening samples. For example, a model may obtain a high UTMOS score because the waveform is smooth while still mispronouncing Filipino words; conversely, a model may improve WER while sounding less natural.

4.7.4 Metric Reliability and Results

The most reliable comparison is a controlled within-study comparison rather than an absolute claim of model quality. Both base and fine-tuned models are evaluated on the same Filipino test prompts, reference voices, inference parameters, ASR model, speaker encoder, UTMOS checkpoint, and text normalization rules. Weights and Biases stores the metric summaries, raw metric logs, generated audio examples, and checkpoint metadata. This makes it possible to compare hyperparameter runs by both development loss and downstream audio metrics.

Table 5

Full PLD Filipino test-set objective results. Lower WER is better; higher SIM-o and UTMOS are better.

Model	Selection	WER (%)	Δ WER	SIM-o	UTMOS
Base OmniVoice	pretrained base	22.55	0.00	0.602	3.64
Fine-tuned, 1000 steps, lr 2e-5	checkpoint-1000	20.07	-2.48	0.610	3.60
Fine-tuned, 2000 steps, lr 5e-6	checkpoint-2000	22.64	+0.09	0.611	3.57
Fine-tuned, lr 1e-5	best eval loss at step 4900	18.52	-4.03	0.604	3.61

Table 5 shows that fine-tuning can improve Filipino intelligibility, but not every fine-tuning run improves the downstream metrics. The 1000-step checkpoint reduced WER from 22.55% to 20.07%, while increasing SIM-o from 0.602 to 0.610 and lowering UTMOS slightly from 3.64 to 3.60. The 2000-step checkpoint trained with a lower learning rate of 5e-6 achieved the highest SIM-o at 0.611, but its WER rose to 22.64%, slightly worse than the base model. This indicates that better speaker similarity alone is not enough to identify the best model for this research question.

The strongest checkpoint is the 1e-5 run selected by best development loss at step 4900. It reduced WER to 18.52%, a 4.03 absolute point improvement over the base model and about a 17.9% relative WER reduction. Its SIM-o of 0.604 remains slightly above the base model, while its UTMOS of 3.61 is lower than the base model but close to the other fine-tuned checkpoints. Therefore, the best current model is the best-development-loss checkpoint because it gives the largest intelligibility improvement without a large speaker-similarity or naturalness collapse.

Table 6

WER edit-operation counts on the full PLD Filipino test set.

Model	Insertions	Deletions	Substitutions	Words
Base OmniVoice	2,668	640	2,858	27,342
Fine-tuned, 1000 steps	2,439	598	2,452	27,354
Fine-tuned, 2000 steps	3,115	586	2,492	27,354
Fine-tuned, best checkpoint	2,045	603	2,417	27,354

The edit-operation counts in Table 6 help explain the model ranking. The best checkpoint substantially reduces insertions and substitutions relative to the base model. In contrast, the 2000-step checkpoint lowers deletions and substitutions but increases insertions enough to lose the WER improvement.

References

- Azzuni, H., and El Saddik, A. (2025). Voice cloning: Comprehensive survey. *arXiv preprint arXiv:2505.00579*. <https://arxiv.org/abs/2505.00579>
- Wang, C., Chen, S., Wu, Y., Zhang, Z., Zhou, L., Liu, S., Chen, Z., Liu, Y., Wang, H., Li, J., He, L., Zhao, S., and Wei, F. (2023). Neural codec language models are zero-shot text to speech synthesizers. *arXiv preprint arXiv:2301.02111*. <https://arxiv.org/abs/2301.02111>
- Le, M., Vyas, A., Shi, B., Karrer, B., Sari, L., Moritz, R., Williamson, M., Manohar, V., Adi, Y., Mahadeokar, J., et al. (2023). Voicebox: Text-guided multilingual universal speech generation at scale. *Advances in Neural Information Processing Systems*, 36, 14005–14034. <https://arxiv.org/abs/2306.15687>

- Wang, Y., Zhan, H., Liu, L., Zeng, R., Guo, H., Zheng, J., Zhang, Q., Zhang, X., Zhang, S., and Wu, Z. (2024). MaskGCT: Zero-shot text-to-speech with masked generative codec transformer. *arXiv preprint arXiv:2409.00750*. <https://arxiv.org/abs/2409.00750>
- Chen, Y., Niu, Z., Ma, Z., Deng, K., Wang, C., Zhao, J., Yu, K., and Chen, X. (2024). F5-TTS: A fairytale that fakes fluent and faithful speech with flow matching. *arXiv preprint arXiv:2410.06885*. <https://arxiv.org/abs/2410.06885>
- Zhu, H., Ye, L., Kang, W., Yao, Z., Guo, L., Kuang, F., Han, Z., Zhuang, W., Lin, L., and Povey, D. (2026). OmniVoice: Towards omnilingual zero-shot text-to-speech with diffusion language models. *arXiv preprint arXiv:2604.00688*. <https://arxiv.org/abs/2604.00688>
- Boson AI. (2025). Higgs Audio V2: Redefining expressiveness in audio generation. GitHub repository and release blog. <https://github.com/boson-ai/higgs-audio>
- Cajote, R. D., Guevara, R. C. L., Bayona, M. G. A. R., and Lucas, C. R. G. (2024). Philippine Languages Database: A multilingual speech corpora for developing systems for Philippine spoken languages. *Proceedings of SIGUL 2024*, 264–271. <https://aclanthology.org/2024.sigul-1.32/>
- Pan, S. J., and Yang, Q. (2010). A survey on transfer learning. *IEEE Transactions on Knowledge and Data Engineering*, 22(10), 1345–1359. <https://doi.org/10.1109/TKDE.2009.191>
- Levenshtein, V. I. (1966). Binary codes capable of correcting deletions, insertions and reversals. *Soviet Physics Doklady*, 10, 707–710.
- Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., and Sutskever, I. (2022). Robust speech recognition via large-scale weak supervision. *arXiv preprint arXiv:2212.04356*. <https://arxiv.org/abs/2212.04356>
- Hsu, W.-N., Bolte, B., Tsai, Y.-H. H., Lakhota, K., Salakhutdinov, R., and Mohamed, A. (2021). HuBERT: Self-supervised speech representation learning by masked prediction of hidden units. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 29, 3451–3460. <https://arxiv.org/abs/2106.07447>
- Gao, Z., Zhang, S., McLoughlin, I., and Yan, Z. (2022). Paraformer: Fast and accurate parallel transformer for non-autoregressive end-to-end speech recognition. *Proceedings of Interspeech 2022*. <https://arxiv.org/abs/2206.08317>
- Desplanques, B., Thienpondt, J., and Demuynck, K. (2020). ECAPA-TDNN: Emphasized channel attention, propagation and aggregation in TDNN based speaker verification. *Proceedings of Interspeech 2020*. <https://arxiv.org/abs/2005.07143>
- Chen, S., Wang, C., Chen, Z., Wu, Y., Liu, S., Chen, Z., Li, J., Kanda, N., Yoshioka, T., Xiao, X., et al. (2021). WavLM: Large-scale self-supervised pre-training for full stack speech processing. *IEEE Journal of Selected Topics in Signal Processing*, 16(6), 1505–1518. <https://arxiv.org/abs/2110.13900>
- Saeki, T., Xin, D., Nakata, W., Koriyama, T., Takamichi, S., and Saruwatari, H. (2022). UTMOS: UTokyo-SaruLab system for VoiceMOS Challenge 2022. *Proceedings of Interspeech 2022*. <https://arxiv.org/abs/2204.02152>
- Rajbhandari, S., Rasley, J., Ruwase, O., and He, Y. (2019). ZeRO: Memory optimizations toward training trillion parameter models. *arXiv preprint arXiv:1910.02054*. <https://arxiv.org/abs/1910.02054>
- Biewald, L. (2020). Experiment tracking with Weights and Biases. Software available from Weights and Biases. <https://www.wandb.com/>